EventEmitter class

1. Definition

The EventEmitter class in Node.js is used to create, handle, and manage events. It allows one part of a program to generate an event and another part to respond to that event.It is available in the built-in events module.

2. Explanation

- **Node.js supports event-driven programming.**
- **The EventEmitter class is used to create and handle custom events.**
- **The emit() method is used to trigger an event.**
- **The on() method is used to register a listener for an event.**
- **When an event occurs, the registered listener function is executed.**
- **It is useful for handling asynchronous operations and notifications.**

3. Important Methods

Method	Description
on()	Registers an event listener.
emit()	Triggers an event.
once()	Executes the listener only once.
removeListener()	Removes a specific listener.
removeAllListeners()	Removes all listeners for an event or all events.

4. Example

```
const EventEmitter = require('events');
const event = new EventEmitter();
event.on('greet', () => {
    console.log('Hello Student!');    });
event.emit('greet');
```

Output:

Hello Student!

5. Advantages

1. **It supports event-driven programming.**
2. **It makes event handling easy.**
3. **It allows different parts of a program to communicate.**
4. **It supports custom events.**
5. **It helps organize application code.**

Methods and Events of EvenEmitter Class

1. Definition

The EventEmitter class in Node.js is used to create, trigger, and handle events. It is available in the built-in events module.

It provides different methods to register event listeners, trigger events, and remove listeners.

2. Important Methods of EventEmitter

1. **on() – Registers a listener for an event.**
2. **emit() – Triggers an event.**
3. **once() – Executes a listener only once.**
4. **removeListener() – Removes a specific event listener.**
5. **removeAllListeners() – Removes all listeners for an event or all events.**
6. **listenerCount() – Returns the number of listeners registered for an event.**

3. Events of EventEmitter

An event is an action or occurrence that can be detected by a program.

Common event-related concepts include:

- **Custom Event: An event created by the programmer, such as login or greet.**
- **Event Listener: A function that executes when an event occurs.**
- **newListener: An event emitted when a new listener is added.**
- **removeListener: An event emitted when a listener is removed.**

4. Example

```
const EventEmitter = require('events');
const event = new EventEmitter();
event.on('greet', () => {
   console.log('Hello Student!'); });
event.emit('greet');
```

Output:

Hello Student!

Explanation:

- **EventEmitter is imported from the events module.**
- **on() registers a listener for the greet event.**
- **emit() triggers the event.**
- **The listener executes and prints the message.**

5. Advantages

1. **It makes event handling easy.**
2. **It supports event-driven programming.**
3. **It allows communication between different parts of a program.**
4. **It allows listeners to be added and removed.**
5. **It makes code more organized.**

6. Conclusion

The methods and events of the EventEmitter class help Node.js applications manage events efficiently. Methods such as on(), emit(), and once() are commonly used for event handling.

Returning event emitter

1. Definition

In Node.js, Returning EventEmitter means returning an EventEmitter object from a function so that events can be registered and handled outside that function.
It allows a function to generate events and lets other parts of the program listen for those events.

2. Explanation

- **The EventEmitter class is imported from the events module.**
- **An EventEmitter object is created inside a function.**
- **The function triggers an event using the emit() method.**
- **The EventEmitter object is returned from the function.**
- **The calling code can register listeners to handle events.**

3. Example

```
const EventEmitter = require('events');
function createEvent() {
    const event = new EventEmitter();
    setTimeout(() => {
        event.emit('greet');
    }, 1000);
    return event;   }
const myEvent = createEvent();
myEvent.on('greet', () => {
    console.log('Hello Student!');
});
Output (after 1 second):
Hello Student!
```

5. Advantages

1. **It allows events to be handled outside a function.**
2. **It improves code reusability.**
3. **It separates event generation from event handling.**
4. **It supports event-driven programming.**

6. Conclusion

Returning an EventEmitter allows a function to return an event object that other parts of a program can use to listen for and handle events. It makes event handling more flexible and organized.

Handle Events only Once

Definition: In Node.js, the once() method is used to execute an event listener only one time, even if the event is triggered multiple times.

Example:

```
const EventEmitter = require('events');
const event = new EventEmitter();
event.once('greet', () => {
    console.log('Hello Student!');   });
event.emit('greet');
event.emit('greet');
```

Output: Hello Student!

Explanation: The event is emitted twice, but the listener executes only once because the once() method is used.

3. Advantages

1. **once() prevents a listener from running repeatedly.**
2. **Error listeners help handle errors properly.**
3. **Both features make applications more reliable.**

Extend EventEmitter Class

1. Definition

In Node.js, extending the EventEmitter class means creating a new class that inherits the properties and methods of the built-in EventEmitter class.

It allows us to create our own classes that can generate and handle custom events.

2. Explanation

- **The EventEmitter class is available in the built-in events module.**
- **The extends keyword is used to inherit the EventEmitter class.**
- **The child class can use methods like on() and emit().**
- **The emit() method triggers an event, while on() registers an event listener.**
- **It is useful for creating reusable event-based components.**

3. Example

```
const EventEmitter = require('events');
class MyEmitter extends EventEmitter {
    greet() {
        this.emit('welcome'); } }
const obj = new MyEmitter();
obj.on('welcome', () => {
    console.log('Welcome Student!'); });
obj.greet();
Output:  Welcome Student!
```

5. Advantages

1. **It allows code reusability.**
2. **It makes event handling easier.**
3. **It supports custom events.**
4. **It keeps code organized.**
5. **It is useful for developing event-driven applications.**

6. Conclusion = Extending the EventEmitter class allows developers to create custom classes that support event handling. It makes Node.js applications more flexible, reusable, and organized.

Error Events

Definition: In Node.js, an error event is used to report errors during program execution. The EventEmitter class uses the special event name 'error' to handle errors.

Example:

```
const EventEmitter = require('events');
const event = new EventEmitter();
event.on('error', (err) => {
    console.log('Error:', err.message); });
event.emit('error', new Error('Something went wrong'));
```

Output: Error: Something went wrong

Explanation: The emit() method triggers the error event and passes an error object to the listener. The listener displays the error message.

Important: If an EventEmitter emits an 'error' event without a registered error listener, Node.js normally throws the error and may terminate the program.

3. Advantages

1. **once() prevents a listener from running repeatedly.**
2. **Error listeners help handle errors properly.**
3. **Both features make applications more reliab.**

Passing Arguments and ‘this’ to listeners

1. Definition

In Node.js, EventEmitter allows us to pass arguments to event listeners when an event is triggered. The this keyword refers to the object that emits the event when a normal function is used as a listener.

2. Explanation

- **The emit() method is used to trigger an event.**
- **Arguments can be passed through emit() to the listener.**
- **The listener receives these arguments and uses them in its function.**
- **With a normal function listener, this refers to the EventEmitter object.**
- **With an arrow function, this does not refer to the EventEmitter object automatically.**

3. Example

```
const EventEmitter = require('events');
const event = new EventEmitter();
event.on('greet', function(name) {
    console.log('Hello ' + name);
    console.log(this === event);
});
event.emit('greet', 'Rahul');
Output:
Hello Rahul
true
```

4. Explanation of Example

1. **An EventEmitter object is created.**
2. **The on() method registers a listener for the greet event.**
3. **The emit() method passes 'Rahul' as an argument.**
4. **The listener receives the name and prints Hello Rahul.**
5. **The this keyword refers to the event object.**

5. Advantages

1. **It allows data to be passed between event emitters and listeners.**
2. **It makes event handling flexible.**
3. **It helps reuse listener functions.**
4. **It provides access to the emitter through this in normal function listeners.**

6. Conclusion

Passing arguments and using this in EventEmitter listeners helps Node.js programs share data and handle events effectively.

Asynchronous and Synchronous call

1. Definition

Synchronous calls execute tasks one by one. The next task waits until the current task is completed.

Asynchronous calls allow a task to run without blocking the execution of other code. The result is handled when the task completes.

2. Explanation

Synchronous Calls

- **Tasks execute in sequence.**
- **Each task waits for the previous task to finish.**
- **It may block further execution.**
- **Example: readFileSync().**

Asynchronous Calls

- **Tasks can run without blocking further execution.**
- **The program can continue while waiting for a result.**
- **Results are handled using callbacks, promises, or async/await.**
- **Example: readFile().**

3. Example

```
const fs = require('fs');
// Synchronous call
const data = fs.readFileSync('test.txt', 'utf8');
console.log(data);
console.log('Sync task completed');
// Asynchronous call
fs.readFile('test.txt', 'utf8', (err, data) => {
    if (err) throw err;
    console.log(data); });
console.log('Async task started');
```

Output: Assuming test.txt exists and has no read errors, the synchronous section prints the file content first, followed by Sync task completed. In the asynchronous section, Async task started usually prints before the file content.

4. Difference Between Synchronous and Asynchronous Calls

Synchronous	Asynchronous
Executes tasks in sequence.	Allows other code to continue while waiting.
Can block execution.	Usually does not block execution while waiting for I/O.
Simple to understand.	Useful for handling multiple operations efficiently.
Example: readFileSync()	Example: readFile()

5. Advantages

Synchronous:

1. **Easy to understand and debug.**
2. **Executes tasks in a predictable order.**

Asynchronous:

1. **Improves application responsiveness.**
2. **Handles I/O operations efficiently.**
3. **Helps prevent blocking the Node.js event loop during I/O waits.**

Connection string for database connectivity,

1. Definition

A connection string is a set of information used to connect an application to a database. It contains details such as the database server, username, password, port, and database name.

2. Explanation

- **It provides the details required to connect to a database.**
- **It specifies the database server and database name.**
- **It may include a username and password for authentication.**
- **Node.js uses database drivers or libraries to establish the connection.**
- **The connection string format depends on the database being used.**

3. Example: MySQL Connection

```
First, install the MySQL package:
npm install mysql2
Program:
const mysql = require('mysql2');
const con = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: '1234',
    database: 'college'  });
con.connect((err) => {
    if (err) throw err;
    console.log('Database connected!'); });
Output:  Database connected!
```

Explanation: The program provides the host, username, password, and database name. The connect() method establishes the database connection.

4. Advantages

1. **Makes database connection easier.**
2. **Specifies the required database details.**
3. **Allows applications to communicate with databases.**
4. **Supports database operations such as insert, update, delete, and select.**

5. Conclusion

A connection string provides the necessary information for connecting a Node.js application to a database. It helps the application access and manage database records.

2 MARK

1. Configuring Database in Node.js

Definition = Configuring a database means setting up a connection between a Node.js application and a database such as MySQL.

Explanation

- **First, install Node.js and MySQL.**
- **Install the MySQL package using npm.**
- **Provide the host, username, password, and database name.**
- **Create a connection using createConnection().**
- **Use the connection to perform database operations.**

Example

```
Install the package:
npm install mysql2
const mysql = require('mysql2');
const con = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: '',
    database: 'college' });
con.connect((err) => {
    if (err) throw err;
    console.log('Database connected'); });
```

Output: Database connected

Advantages

1. **Connects Node.js with MySQL.**
2. **Allows database operations.**
3. **Makes data storage and retrieval easier.**

2. Working with INSERT Command

Definition = The INSERT command in MySQL is used to add new records into a table.

Explanation

- **It adds new data to a database table.**
- **We specify the table name and column names.**
- **The VALUES keyword provides the values to insert.**
- **Node.js can execute an INSERT query using the database connection.**

Syntax

```
INSERT INTO student (id, name)
VALUES (1, 'Rahul');
Example in Node.js
const sql = "INSERT INTO student (id, name) VALUES (?, ?)";
con.query(sql, [1, 'Rahul'], (err, result) => {
    if (err) throw err;
    console.log('Record inserted'); });
```

Output: Record inserted

Advantages

1. **Adds new records to a table.**
2. **Stores information permanently in the database.**
3. **Supports inserting multiple records.**

3. Working with SELECT Command

Definition = The SELECT command is used to retrieve data from a database table.

Explanation

- **It displays records stored in a table.**
- **The * symbol selects all columns.**
- **Specific column names can also be selected.**
- **The WHERE clause retrieves records that match a condition.**

Syntax

```
SELECT * FROM student;
Example in Node.js
con.query("SELECT * FROM student", (err, rows) => {
    if (err) throw err;
    console.log(rows); });
```

Output (example): [{ id: 1, name: 'Rahul' }]

Advantages

1. **Retrieves stored records.**
2. **Selects specific columns.**
3. **Filters records using conditions.**
4. **Helps users access required information.**

4. Updating Records

Definition

The UPDATE command in MySQL is used to modify existing records in a database table.

Explanation

- **It changes existing data in a table.**
- **The SET keyword specifies the new value.**
- **The WHERE clause identifies which record should be updated.**
- **Without WHERE, all records may be updated.**

Syntax

```
UPDATE student
SET name = 'Priya'
WHERE id = 1;
```

Example in Node.js

```
const sql = "UPDATE student SET name = ? WHERE id = ?";
con.query(sql, ['Priya', 1], (err, result) => {
    if (err) throw err;
    console.log('Record updated'); });
```

Output: Record updated

1.

Advantages

1. **Modifies existing records.**
2. **Corrects incorrect information.**
3. **Updates selected records using conditions.**

5. Deleting Records

Definition = The DELETE command in MySQL is used to remove records from a database table.

Explanation

- **It removes unwanted records.**
- **The WHERE clause specifies which records to delete.**
- **Without WHERE, all records in the table may be deleted.**
- **The table structure remains unchanged.**

Syntax

```
DELETE FROM student
WHERE id = 1;
Example in Node.js
const sql = "DELETE FROM student WHERE id = ?";
con.query(sql, [1], (err, result) => {
    if (err) throw err;
    console.log('Record deleted');  });
```

Output: Record deleted

Advantages

1. **Removes unwanted records.**
2. **Helps maintain correct data.**
3. **Deletes selected records using conditions.**

6. Drop Tables

Definition = The DROP TABLE command in MySQL is used to permanently remove a table from a database, including its structure and stored records.

Explanation

- **It deletes the complete table.**
- **Both the table structure and its data are removed.**
- **After dropping the table, it no longer exists in the database.**
- **It should be used carefully because the operation can cause data loss.**

Syntax

```
DROP TABLE student;
Example in Node.js
con.query("DROP TABLE student", (err, result) => {
    if (err) throw err;
    console.log('Table dropped');  });
```

Output: Table dropped

Advantages

1. **Removes tables that are no longer required.**
2. **Removes both data and table structure.**
3. **Helps manage database objects.**

Difference Between DELETE and DROP

DELETE	DROP
Removes records.	Removes the complete table.
Table structure remains.	Table structure is removed.
Can use WHERE.	Does not use WHERE.

7. Ordered Result Set (ORDER BY)

Definition = The ORDER BY clause in MySQL is used to arrange records in ascending or descending order based on one or more columns.

Explanation

- **It arranges records in a specific order.**
- **ASC sorts records in ascending order.**
- **DESC sorts records in descending order.**
- **The default order is ascending (ASC).**
- **It is commonly used with the SELECT command.**

Syntax

```
SELECT * FROM studentORDER BY name ASC;
Example in Node.js
const sql = "SELECT * FROM student ORDER BY name ASC";
con.query(sql, (err, rows) => {
    if (err) throw err;
    console.log(rows); });
```

Example output: Suppose the table contains:

id	name
1	Rahul
2	Amit
3	Priya

After sorting by name in ascending order:

id	name
2	Amit
3	Priya
1	Rahul

Advantages

1. **Arranges records systematically.**
2. **Makes data easier to read.**
3. **Supports ascending and descending order.**
4. **Helps find records in a required sequence.**